

Python Functions, Iterators, Generators & Lambda - Solutions

Theory Answers

1. Difference between Function and Method:

A function is defined independently using def keyword. A method is a function associated with an object.

```
def greet():  
    return "Hello"  
  
class Person:  
    def speak(self):  
        return "Hi"
```

2. Function Arguments and Parameters:

Parameters are variables defined in function definition. Arguments are values passed during function call.

```
def add(a, b):    # parameters  
    return a + b  
  
add(5, 3)        # arguments
```

3. Different ways to define and call function:

```
def func(a, b=2):    # default argument  
    return a + b  
  
func(5)  
func(5, 10)
```

4. Purpose of return statement:

```
def square(n):  
    return n*n
```

5. Iterators vs Iterables:

Iterable can be looped over. Iterator implements `__iter__()` and `__next__()`.

```
nums = [1,2,3]  
it = iter(nums)  
print(next(it))
```

6. Generators:

```
def gen():  
    yield 1
```

```
yield 2
```

7. Advantages of Generators:

Generators save memory and generate values on demand.

8. Lambda Function:

```
square = lambda x: x*x
```

9. map() function:

```
nums = [1,2,3]  
result = list(map(lambda x: x*2, nums))
```

10. Difference between map, filter, reduce:

map() transforms, filter() selects, reduce() aggregates values.

11. Internal mechanism of reduce on [47,11,42,13]:

Step1: $47+11=58$

Step2: $58+42=100$

Step3: $100+13=113$

Practical Answers

1. Sum of even numbers:

```
def sum_even(lst):  
    return sum(x for x in lst if x % 2 == 0)
```

2. Reverse string:

```
def reverse_string(s):  
    return s[::-1]
```

3. Squares of list:

```
def squares(lst):  
    return [x*x for x in lst]
```

4. Prime number from 1 to 200:

```
def is_prime(n):  
    if n < 2:  
        return False  
    for i in range(2, int(n**0.5)+1):  
        if n % i == 0:  
            return False  
    return True
```

5. Fibonacci Iterator Class:

```
class Fibonacci:  
    def __init__(self, n):  
        self.n = n  
        self.a, self.b = 0, 1  
        self.count = 0  
  
    def __iter__(self):  
        return self  
  
    def __next__(self):  
        if self.count >= self.n:  
            raise StopIteration  
        self.a, self.b = self.b, self.a + self.b  
        self.count += 1  
        return self.a
```

6. Generator for powers of 2:

```
def powers_of_two(n):  
    for i in range(n+1):  
        yield 2**i
```

7. Generator to read file line by line:

```
def read_file(file):  
    with open(file, 'r') as f:  
        for line in f:  
            yield line
```

8. Lambda sort by second element:

```
lst = [(1,3),(2,1),(4,2)]
lst.sort(key=lambda x: x[1])
```

9. Celsius to Fahrenheit using map:

```
temps = [0, 20, 30]
fahrenheit = list(map(lambda c: (c*9/5)+32, temps))
```

10. Remove vowels using filter:

```
string = "Harsha"
result = ''.join(filter(lambda x: x.lower() not in 'aeiou', string))
```

11. Bookshop order calculation using lambda and map:

```
orders = [
    [34587, "Learning Python", 4, 40.95],
    [98762, "Data Science", 1, 98.99],
    [77226, "AI Book", 3, 24.99]
]

result = list(map(lambda x:
    (x[0], x[2]*x[3] + 10) if x[2]*x[3] < 100
    else (x[0], x[2]*x[3]),
    orders))

print(result)
```